Kazakh-British Technical University

Data collection and preparation

Laboratory work №5

Dynamic Web Scraping & Data Cleaning: Selenium/Playwright automation + missing values, regex string cleaning, Pandera validation.

Full Name:

Baibolov Sultanmakhmut

ID:

23B031235

29.09.2026

Almaty

Table of contents

Task 1. Dynamic data collection 3
Task 2. Missing values 4
Task 3. String cleaning 5
Task 4. Data validation 6
Task 5. Final dataset 7

Task 1. Dynamic data collection

Use Selenium or Playwright to open the selected website and collect data from dynamically loaded content. Extract at least three useful fields from each record and store the collected data in a Pandas DataFrame. Collect data from more than one page, section, or dynamically loaded block.

Task 1. Dynamic data collection

[2]:
```python
BASE_URL = "https://soundcloud.com/search"
```

[3]:
```python
search_queries = ["radiohead", "panchiko", "gorillaz"]
scrolls = 3
timeout_seconds = 2
```

[4]:
```python
def scroll_down(driver: WebDriver) -> None:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
```

[5]:
```python
def extract_track_info(driver: WebDriver, element: WebElement) -> dict:
    record = {
        "type": "",
        "title": "",
        "artist": "",
        "uploaded": "",
        "url": "",
        "plays": -1,
        "comments": -1,
        "likes": -1,
        "reposts": -1
    }
    type_pattern = re.compile(r"^(playlist|track)$")
    try:
        container = element.find_element(By.CLASS_NAME, "searchItem__trackItem")

        for container_class in container.get_attribute("class").split(" "):
            if m := type_pattern.match(container_class):
                record["type"] = m.group(1)

        if record["type"] == "playlist":
            time.sleep(2)

        record["likes"] = container.find_element(By.CSS_SELECTOR, ".sc-button-like").text
        record["reposts"] = container.find_element(By.CSS_SELECTOR, ".sc-button-repost").text
```

[6]:
```python
def parse_soundcloud(driver: WebDriver) -> list[dict] | None:
    result = []
    seen = set()

    try:
        driver = webdriver.Chrome()

        for query in search_queries:

            driver.get(BASE_URL + f"?q={query}")

            WebDriverWait(driver, 10).until(
                EC.presence_of_element_located((By.CLASS_NAME, "searchList__item"))
            )

            for _ in range(scrolls):
                scroll_down(driver)
                time.sleep(3)
                list_elements = driver.find_elements(By.CLASS_NAME, "searchList__item")

                for li_element in list_elements:
                    record = extract_track_info(driver, li_element)

                    if not record["type"]:
                        continue

                    if record["url"] in seen:
                        continue

                    seen.add(record["url"])
                    result.append(record)


    except NoSuchElementException as e:
        pass
    finally:
```

[7]:
```python
def should_fetch(filepath: str) -> bool:
    path = Path(filepath)
    if path.exists() and path.stat().st_size != 0:
        return False
    return True
```

[8]:
```python
if should_fetch("./soundcloud_records.json"):
    records = parse_soundcloud(driver)
    with open("./soundcloud_records.json", "w") as json_file:
        json.dump(records, json_file)
```

[9]:
```python
df = pd.read_json("./soundcloud_records.json")
```

[10]:
```python
df.duplicated().sum()
```

[10]:
```
np.int64(0)
```

Task 2. Missing values

[11]:
```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 62 entries, 0 to 61
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   type      62 non-null     str
 1   title     62 non-null     str
 2   artist    62 non-null     str
 3   uploaded  62 non-null     str
 4   url       62 non-null     str
 5   plays     62 non-null     object
 6   comments  62 non-null     object
 7   likes     62 non-null     str
 8   reposts   62 non-null     str
dtypes: object(2), str(7)
memory usage: 11.3+ KB
```

I used Selenium to get a list of tracks and playlists from the SoundCloud website. I scraped 3 search pages with the values Radiohead, Pachiko, and Gorillaz. I created functions for parsing the track card separately, for scrolling through the website, and a general one with the creation of a Chrome driver and switching between screens after parsing each. The result was a JSON file with raw data and a DataFrame.

Task 2. Missing values

Inspect the collected dataset and identify missing values. Process the missing values using appropriate methods such as removing records, filling values, or keeping them unchanged when justified. Show the number of missing values before and after processing.

[15]:
```python
df["comments"] = df["comments"].replace(
    "-1",
    pd.NA
)
df["plays"] = df["plays"].replace(
    "-1",
    pd.NA
)
```

[16]:
```python
df["likes"] = df["likes"].replace(
    "",
    pd.NA
)
df["reposts"] = df["reposts"].replace(
    "",
    pd.NA
)
```

[17]:
```python
df.head()
```

[17]:

	type	title	artist	uploaded	url	plays	comments	likes	reposts
0	track	Radiohead - Creep (slowed)	A	2018-05-27T21:26:36.000Z	https://soundcloud.com/user-33590647	7,544,087 plays	1,657 comments	101K	810
1	playlist	Radiohead	...	2025-06-02T00:44:23.000Z	https://soundcloud.com/raian-cma	NaN	NaN	NaN	NaN
2	track	Let Down - Radiohead	user618723838	2026-03-30T11:09:23.000Z	https://soundcloud.com/user618723838	737,211 plays	136 comments	14K	71
3	track	Radiohead - No Suprises	Pump Glock	2017-03-16T23:06:56.000Z	https://soundcloud.com/pumpglock	3,647,717 plays	1,715 comments	55.2K	440
4	track	Radiohead - Karma Police (1997)	skintowear	2023-11-18T10:08:22.000Z	https://soundcloud.com/skintowear	3,774,697 plays	928 comments	56.9K	340

[18]:
```python
df.isna().sum()
```

1	playlist	Radiohead	...	2025-06-02T00:44:23.000Z	https://soundcloud.com/raian-cma	NaN	NaN	NaN	NaN
2	track	Let Down - Radiohead	user618723838	2026-03-30T11:09:23.000Z	https://soundcloud.com/user618723838	737,211 plays	136 comments	14K	71
3	track	Radiohead - No Suprises	Pump Glock	2017-03-16T23:06:56.000Z	https://soundcloud.com/pumpglock	3,647,717 plays	1,715 comments	55.2K	440
4	track	Radiohead - Karma Police (1997)	skintowear	2023-11-18T10:08:22.000Z	https://soundcloud.com/skintowear	3,774,697 plays	928 comments	56.9K	340

[18]:
```
df.isna().sum()
```

[18]:
```
type        0
title       0
artist      0
uploaded    0
url         0
plays       5
comments    5
likes       4
reposts     5
dtype: int64
```

[19]:
```
df = df.dropna()
```

[20]:
```
df.isna().sum()
```

[20]:
```
type        0
title       0
artist      0
uploaded    0
url         0
plays       0
comments    0
likes       0
reposts     0
dtype: int64
```

First of all, I normalized all the missing values to the pandas object so that they could be calculated and deleted or filled in accordingly. Since all the cisl columns of the playlists were skipped, of which there were only 5, I decided to delete the rows with the missing values. After checking, there were no more missing values.

Task 3. String cleaning

Select text columns that require cleaning and process them using Pandas string methods and regular expressions. Remove unnecessary spaces or symbols, standardize text format, and extract useful information from strings where appropriate. Examples may include extracting numbers, prices, dates, ratings, codes, or units from text.

[23]:
```
df["type"] = df["type"].astype("category")
```

[24]:
```
df["plays"] = df["plays"].str.replace(r"\D", "", regex=True)
df["plays"] = df["plays"].astype("int64")
df["comments"] = df["comments"].str.replace(r"\D", "", regex=True)
df["comments"] = df["comments"].astype("int64")
df["reposts"] = df["reposts"].str.replace(r"\D", "", regex=True)
df["reposts"] = df["reposts"].astype("int64")
```

[25]:
```
df.head()
```

[25]:

	type	title	artist	uploaded	url	plays	comments	likes	reposts
0	track	Radiohead - Creep (slowed)	A	2018-05-27T21:26:36.000Z	https://soundcloud.com/user-33590647	7544087	1657	101K	810
2	track	Let Down - Radiohead	user618723838	2026-03-30T11:09:23.000Z	https://soundcloud.com/user618723838	737211	136	14K	71
3	track	Radiohead - No Suprises	Pump Glock	2017-03-16T23:06:56.000Z	https://soundcloud.com/pumpglock	3647717	1715	55.2K	440
4	track	Radiohead - Karma Police (1997)	skintowear	2023-11-18T10:08:22.000Z	https://soundcloud.com/skintowear	3774697	928	56.9K	340
5	track	radiohead all i need	babypinkbehelit	2022-02-26T00:06:14.000Z	https://soundcloud.com/mazziedstar	3538216	932	54.3K	358

[26]:
```
likes = df["likes"].str.replace(",", "")
parts = likes.str.extract(r"([\d.]+)(K)?")
num = pd.to_numeric(parts[0])
mult = parts[1].map({"K": 1000}).fillna(1)
df["likes"] = (num * mult)
df["likes"] = df["likes"].astype("int64")
```

[27]:
```
df["uploaded"] = pd.to_datetime(df["uploaded"])
```

[28]:
```
df.head()
```

[28]:

	type	title	artist	uploaded	url	plays	comments	likes	reposts
0	track	Radiohead - Creep (slowed)	A	2018-05-27 21:26:36+00:00	https://soundcloud.com/user-33590647	7544087	1657	101000	810

```
parts = likes.str.extract(r'([\d.]+)(K)?')
num = pd.to_numeric(parts[0])
mult = parts[1].map({"K": 1000}).fillna(1)
df["likes"] = (num * mult)
df["likes"] = df["likes"].astype("int64")
```

[27]:
```
df["uploaded"] = pd.to_datetime(df["uploaded"])
```

[28]:
```
df.head()
```

[28]:

	type	title	artist	uploaded	url	plays	comments	likes	reposts
0	track	Radiohead - Creep (slowed)	A	2018-05-27 21:26:36+00:00	https://soundcloud.com/user-33590647	7544087	1657	101000	810
2	track	Let Down - Radiohead	user618723838	2026-03-30 11:09:23+00:00	https://soundcloud.com/user618723838	737211	136	14000	71
3	track	Radiohead - No Suprises	Pump Glock	2017-03-16 23:06:56+00:00	https://soundcloud.com/pumpglock	3647717	1715	55200	440
4	track	Radiohead - Karma Police (1997)	skintowear	2023-11-18 10:08:22+00:00	https://soundcloud.com/skintowear	3774697	928	56900	340
5	track	radiohead all i need	babypinkbehelit	2022-02-26 00:06:14+00:00	https://soundcloud.com/mazziedstar	3538216	932	54300	358

[29]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 56 entries, 0 to 61
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   type      56 non-null     category
 1   title     56 non-null     str
 2   artist    56 non-null     str
 3   uploaded  56 non-null     datetime64[us, UTC]
 4   url       56 non-null     str
 5   plays     56 non-null     int64
 6   comments  56 non-null     int64
 7   likes     56 non-null     int64
 8   reposts   56 non-null     int64
dtypes: category(1), datetime64[us, UTC](1), int64(4), str(3)
memory usage: 8.3 KB
```

I used the replace methods to simply replace all characters with empty strings, except for numeric ones. For the likes column, I had to write more complex logic because the values contained characters. I split the string into a numeric part and the multiplier values. Then I extracted both parts, converted them, and combined them back. I converted each column to the appropriate data type, including categories, strings, datetime, and int.

Task 4. Data validation

Create a validation schema using Pandera for the cleaned dataset. Define rules for several columns, including expected data types and suitable conditions such as allowed values, value ranges, nonnull fields. Run the validation and correct detected problems if necessary.

Task 4. Data validation

[30]:
```
schema = pa.DataFrameSchema({
    "type": pa.Column(pa.Category, pa.Check.isin(["track", "playlist"])),
    "title": pa.Column(pa.String),
    "artist": pa.Column(pa.String),
    "uploaded": pa.Column(pa.DateTime, coerce=True),
    "url": pa.Column(pa.String, pa.Check.str_startswith("https://soundcloud.com/"), unique=True),
    "plays": pa.Column(pa.Int64, pa.Check.greater_than_or_equal_to(0)),
    "comments": pa.Column(pa.Int64, pa.Check.greater_than_or_equal_to(0)),
    "likes": pa.Column(pa.Int64, pa.Check.greater_than_or_equal_to(0)),
    "reposts": pa.Column(pa.Int64, pa.Check.greater_than_or_equal_to(0)),
})
```

[31]:
```
try:
    validated_df = schema.validate(df)
except pa.errors.SchemaErrors as err:
    print(err.failure_cases)
```

[32]:
```
validated_df.head()
```

[32]:

	type	title	artist	uploaded	url	plays	comments	likes	reposts
0	track	Radiohead - Creep (slowed)	A	2018-05-27 21:26:36	https://soundcloud.com/user-33590647	7544087	1657	101000	810
2	track	Let Down - Radiohead	user618723838	2026-03-30 11:09:23	https://soundcloud.com/user618723838	737211	136	14000	71
3	track	Radiohead - No Suprises	Pump Glock	2017-03-16 23:06:56	https://soundcloud.com/pumpglock	3647717	1715	55200	440
4	track	Radiohead - Karma Police (1997)	skintowear	2023-11-18 10:08:22	https://soundcloud.com/skintowear	3774697	928	56900	340
5	track	radiohead all i need	babypinkbehelit	2022-02-26 00:06:14	https://soundcloud.com/mazziedstar	3538216	932	54300	358

Task 5. Final dataset

[33]:
```
final_df = validated_df
```

I validated the resulting DataFrame using a simple scheme to check whether the overall data complies with the criteria for each column. Thanks to this step, I was able to notice that the data type of the “uploaded” column was in a non-standard format for datetime according to

the ISO standard. Pandera converted it to the required format on its own.

Task 5. Final dataset

Prepare the final cleaned dataset. Check missing values, duplicates, data types, and validation results. Compare the original scraped data with the cleaned version and write five findings describing the main data-quality problems and changes made during cleaning.

[36]:
```
final_df.info()
```

```
<class 'pandas.DataFrame'>
Index: 56 entries, 0 to 61
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   type      56 non-null     category
 1   title     56 non-null     str
 2   artist    56 non-null     str
 3   uploaded  56 non-null     datetime64[ns]
 4   url       56 non-null     str
 5   plays     56 non-null     int64
 6   comments  56 non-null     int64
 7   likes     56 non-null     int64
 8   reposts   56 non-null     int64
dtypes: category(1), datetime64[ns](1), int64(4), str(3)
memory usage: 8.4 KB
```

[37]:
```
final_df.to_csv("./soundcloud_dataset.csv", index=False)
```

Findings

1. **Missing values:** 5 missing cells in the uncleaned df within numerical columns (plays, comments, likes, reposts). They were deleted, as there are no information that allows to fill them
2. **Numerical columns stored as text:** values like `1.2M`, `34K` and `1,234` could not be analyzed. They were parsed with regex string replacing
3. **Date column:** The datetime format stored in soundcloud webpage was with miccroseconds, pandera scheme transformed it to nanoseconds as in ISO format.
4. **Different missing data values:** Missing values in likes and reposts had empty string whereas plays and comments had -1 value represeting NaN. They were casted to `pd.NA`.
5. **Duplicates and validation:** The intial json file didn't contain any dulpicates as they have been removed during fetching process.